2025 34th International Conference on Parallel Architectures and Compilation Techniques (PACT) | 979-8-3315-8295-1/25/$31.00 ©2025 IEEE | DOI: 10.1109/PACT65351.2025.00036

Bancroft: Genomics Acceleration Beyond On-Device Memory

Se-Min Lim
College of Computer Science
Kookmin University
Seoul, South Korea
seminl1@kookmin.ac.kr

Seongyoung Kang
Department of Computer Science
University of California, Irvine
Irvine, CA, USA
seongyk3@uci.edu

Sang-Woo Jun
Department of Computer Science
University of California, Irvine
Irvine, CA, USA
swjun@ics.uci.edu

***Abstract*—This paper presents Bancroft, a computational genomics acceleration platform for processing datasets far exceeding accelerator memory capacity. Bancroft overcomes the capacity limitations of accelerator memory by storing genomic data in a novel genomic compression format on the host server, and decompressing it on-demand within the accelerator after fetching it over PCIe. The key innovation of Bancroft is algorithmic optimizations for reference-based compression and decompression of popular genomic file formats. The algorithm achieves high enough compression ratios to improve the effective bandwidth of the PCIe to DRAM-levels, while facilitating highly efficient hardware implementations. We evaluate a prototype implementation of Bancroft on an affordable Alveo U50 FPGA accelerator card equipped with 8 GB of High-Bandwidth Memory (HBM). Our evaluation demonstrates that Bancroft delivers speeds exceeding on-device DDR4 memory and over 30% of HBM performance, while incurring only 30% chip space overhead. This is an order of magnitude higher performance and efficiency compared to conventional PCIe-limited architectures. Using a real-world pre-alignment filtering application, Bancroft demonstrates over 7× performance improvement over conventional accelerators on scalable datasets.**

***Index Terms*—Genomics, Acceleration, Compression, HBM**

I. Introduction

The rapid advancement of genome sequencing technologies has led to an explosion of collected genomic data, projected to become one of the largest and computationally demanding areas of computing in the next decade [1]. Genome sequencing costs have reduced exponentially [2], leading to an exponential increase in the sizes of genomic databases [3]. In fact, the projected growth of genomic data is expected to outpace even Moore's law, growing faster than the rates of both computational power and memory capacity scaling [4]–[6].

Hardware accelerators such as Graphic Processing Units (GPUs) and Field-Programmable Gate Arrays (FPGAs) are promising solutions to such scalability problems, since it is unclear whether performance and capacity scaling of conventional CPU and DRAM are capable of keeping up with future demands [7]–[11]. Accelerators can supply high computational throughput and power efficiency via optimized hardware exploiting massive parallelism and pipelining [12]–[15]. They are also typically equipped with high-performance memory fabric, such as High-Bandwidth Memory (HBM), to satisfy both the computational and data access performance requirements of genomics applications [6].

Unfortunately, such accelerators often end up being underutilized with large problems with real-world scale, due to the capacity limitations of on-board memory [16]–[19]. Accelerators are often equipped with fast, but relatively small on-board memory, typically in the order of tens of GBs. If the working set of an algorithm exceeds this capacity, data accesses must be serviced over either slow PCIe or network, resulting in orders of magnitude performance degradation. Many important genomics applications fall into this category, including long read sequence alignment, pre-alignment filtering, overlap graph construction, and more [20], [21].

A. Bancroft: Scalability via Compression

In this work, we address the memory scalability issue with **Bancroft**, a genomics accelerator platform which provides DRAM-level random-access bandwidth to scalable genomics datasets beyond on-device memory capacity. Bancroft achieves this with a novel, hardware-codesigned genome compression algorithm, which achieves high compression ratios, low hardware resource overhead, and fast throughput.

Bancroft's compression algorithm handles the two most popular genome data formats: FASTA and FASTQ. The FASTA format stores only the sequences of nucleotide bases, or "base reads" (`a,c,g,t`). We compress this information using a novel variant of reference-based genome compression algorithms [22], [23], optimized with features which facilitate efficient hardware implementation. The FASTQ format expands FASTA to include a single-byte quality score associated with each base. Bancroft implements a bitmask-based compression scheme to achieve both high compression ratios and throughput. The compression scheme is designed so that most of the compression and decompression hardware can be efficiently reused across the two algorithms. Section III describes the algorithmic optimizations in detail, including fixed-width grouped header encoding, fixed-stride exact matching, and wide mismatch strides.

The hardware platform works together with a host-side software management layer, which maintains the data structures that support random access into the genome data, as well as orchestrates the data movement between the compressed

979-8-3315-8295-1/25/$31.00 ©2025 IEEE
DOI 10.1109/PACT65351.2025.00036

genome database and the user-provided application-specific hardware accelerator kernel.

B. Evaluation and Results

We evaluate Bancroft with an FPGA prototype implemented using the Xilinx Alveo U50 board, tested with various genomic workloads including pre-alignment filtering, using long read and reference genomes from multiple different organisms. The U50 is a useful and representative evaluation platform thanks to its realistic PCIe Gen 3×16 interface and 3D-stacked HBM memory, both of which are the typical limiting factors in data-intensive analytics. It is also useful as a deployment target for Bancroft by itself, since the U50 is a mid-range FPGA affordable to many potential users. Furthermore, FPGAs have often demonstrated superior performance and power efficiency on many representative genomics-related workloads, compared to other programmable accelerators such as GPUs [24]–[28].

Bancroft on the FPGA prototype can provide a user kernel with practically unlimited memory capacity, at an order of magnitude higher resource efficiency compared to the state-of-the-art. An application-specific accelerator built on Bancroft can access genomic data over a multi-channel interface similar to HBM. Bancroft is able to instantiate up to ten I/O channels on the U50 FPGA, limited only by the number number of channels supported by its HBM interface. Each channel delivers close to 16 GB/s of genomic data bandwidth, tested with both long-read datasets and pre-assembled references. This adds up to 160 GB/s of effective bandwidth within PCIe bandwidth limitations, equivalent to 31% of the peak HBM performance. Bancroft achieves this performance while consuming only 30% of the chip space. Such high efficiency is made possible by the hardware co-optimized compression algorithm achieving high compression ratios between 24× and 100+× on FASTA, and between 4.7× and 7.4× on FASTQ formats, showing ratios competitive with state-of-the-art schemes including SPRING [29], CRAM [30], and CoLoRd [31]. Bancroft also achieves high compression throughput of up to 3.7 GB/s per channel, an order of magnitude faster than any existing FPGA genomic compression accelerator.

We demonstrate the overall system performance improvements of a real-world workload of pre-alignment filtering for long-read alignment. Bancroft demonstrates over 7× performance improvements over conventional accelerators integrated into a realistic system, whose performance is limited by the PCIe bandwidth.

C. Contribution and Significance

In summary, Bancroft claims the following contributions:

- A novel hardware-optimized genome compression algorithm capable of state-of-the-art compression ratios, while enabling efficient hardware implementation and facilitating high throughput, for both bases and quality scores.
- The design and implementation of an efficient hardware architecture for the compression algorithm, achieving an order of magnitude improvement in both performance and resource utilization compared to the state-of-the-art.
- Detailed evaluation of the algorithm and overall system, using various genomic workflows as well as a real-world pre-alignment filtering application.

Bancroft is capable of handling large genomics problems even if CPU performance and DRAM capacity scaling become difficult in the near future [7], [9]–[11], by delivering DRAM-level bandwidth into TB-scale datasets to hardware accelerators, while using a fixed amount of HBM.

II. Background and Related Works

A. Computational Genomics

The increasingly wide availability of personal genomic data, coupled with rapid algorithmic advancements related to computational genomics, are enabling many important and useful applications, including personalized and precision medicine [32], wide-scale genome-wide association studies [33], [34] for disease analysis, future agriculture [35], and tracing and managing pandemics [36], [37]. Emerging solutions suggest that highly effective personalized therapies for previously untreatable diseases may become available in the near future [38], [39].

The fundamental task of computational genomics is to extract insight from genomic data collected using modern sequencing technologies. Because we cannot simply read genomic data from cell samples in a single complete string yet, what sequencing machines do is generate vast amounts of randomly sampled substrings from the whole genome. These substrings are called “reads”, and we cannot know where they were sampled from. They also contain noise and other probabilistic errors. Based on the average length of reads, sequencing technologies are categorized into short reads (hundreds or fewer) or long reads (thousands to millions). Furthermore, since the DNA is a double helix consisting of two complementary strands, a read may come from either strand, resulting in either a “forward” read, or a “reverse complement” read. A forward read can be computed from the reverse read and vice versa, and the existence of opposite directions in the sample must be considered during analysis.

The first task of computational genomics is typically to reconstruct the reads into a whole genome while removing noise and errors, typically using numerous overlapping samples, in a process called “sequence assembly”. Assembly can be done using a pre-assembled reference genome of each species to take advantage of similarities between individuals (“reference-based” assembly) or just between the reads themselves “De Novo” assembly), either because a reference is not available, or to avoid errors such as “reference drift” [40], [41]. Both approaches depend on “sequence alignment”, discovering locations with similar substrings.

B. Genomics and Acceleration

Since genome analysis is becoming essential in various fields of human life, while the price of getting genomic sequence data is decreasing, it is becoming extremely difficult

to analyze the enormous size of the genome sequence data without acceleration. Many existing research has introduced acceleration to sequence alignment, read mapping, classification, and structure similarities for faster analysis using various acceleration platforms, such as GPUs [42]–[44], FPGAs [45]–[48], ASICs [49]–[52], PIMs [5], [53]–[55], quantum computing [56], and in-storage processing [57].

Among the acceleration platforms, GPUs and FPGAs are the most popular platforms due to their affordability and off-the-shelf availability. Thanks to the capability of composing fine-grained application-specific reconfigurable logic, FPGAs often show better performance and power efficiency than GPU-based accelerators, especially for the representative genome analysis algorithm, sequence alignment [58]–[60].

C. Memory and Storage Scalability

Along with the looming end of Moore's law, the future of memory scalability is also uncertain. Density of capacitor-based DRAM in the 2D space is expected to stop scaling, with no feasible alternative [61]. One direction of continued improvement is through 3D stacking. For memories such as High Bandwidth Memory (HBM), bandwidth continues to scale with ever-wider communication bandwidth using Through-Silicon Vias (TSV). The aggregate bandwidth of HBM is so high that it is spread across multiple independent pseudo-channels (PC) to avoid prohibitively wide buses. However, while bandwidth continues to scale, the height of 3D stacking faces physical limitations, making capacity scaling more uncertain [8]–[11]. This is a concerning prospect as the capacity scale of problems such as computational genomics continues to grow. As a result, heterogeneous systems combining DRAM, NVMe, and more are under active investigation [44], [62], as well as memory disaggregated over a network, using emerging fabrics such as CXL [63], [64].

D. Genome Compression

As the scale of genomic data continues to grow, compression is becoming more important to reduce both data archiving and transmission. Unfortunately, conventional data-agnostic compression algorithms such as dictionaries and Huffman coding have shown very little compression efficiency due to the high entropy of genomic data [31], [65].

Many genome-specific compression schemes are under active investigation, and they can largely be categorized into two groups. The first group is reference-free compression, where the algorithm only looks at the genome to compress, and takes advantage of statistical trends it can find [29], [31]. These schemes typically require little memory, but achieve low compression ratios. The second group is reference-based compression, which uses a reference, (or "consensus") and takes advantage of the similarity between individuals of the same species to encode only the differences [30], [31]. These schemes often achieve very high compression ratios in the hundreds, but require more memory because the entire reference needs to be memory-resident. We emphasize that a reference needs to be created only once per species, potentially by the developers of the algorithm. The same compression reference can be used by all users of the algorithm, similar to how references are used for sequence alignment and assembly. Approaches beyond these categories are also being explored, including using machine learning models [66]–[68].

The majority of genomic sequences are presented in either FASTA or FASTQ format. While the FASTA format only stores the nucleotide bases of sequence read data (`a,c,g,t`), the FASTQ format also stores the quality score associated with each base. As a result, the compression of quality scores holds significant importance. Advanced FASTQ compression tools support either lossless or lossy methods for the compression of quality scores to aggressively shrink the size of the dataset [29]–[31], [69], [70].

Acceleration of genome compression is also under active investigation to overcome the slow speed of these algorithms, using FPGAs [71], [72], and GPUs [73]–[75].

III. Bancroft Compression Algorithm

The key novelty of Bancroft is its compression algorithm, a reference-based lossless algorithm co-optimized with hardware acceleration to achieve high performance for both compression and decompression, without sacrificing high compression ratios on both base reads and quality scores. Both compression and decompression performance are important goals, for analytics acceleration as well as for archiving and distribution.

The Bancroft compression algorithm belongs to the class of reference-based compression algorithms, which find matching regions between the input "*target*" data and a "*reference*" genome. The matches can be encoded in compact offset-length pairs. We note that the reference used for compression is usually unrelated to the reference used for alignment (if any). We also emphasize that the compression reference only needs to be created once for each species (e.g, one for all humans), which we have already done. All users of the algorithm can use our provided reference to compress and decompress all human genomes.

A. Compressed Encoding Scheme

We first describe the encoding scheme of the genome compression algorithm, since its design is carefully tuned to facilitate efficient hardware implementations. Figure 1 illustrates two encoding schemes, each for base reads and quality scores. Both formats consists of a sequence of 2-bit headers and 32-bit fixed-width payloads. They use the same fixed-width 32-bit grouped header encoding consisting of 16 two-bit headers, similar to Group VarInt [76]. Both grouped headers and fixed-width payloads vastly simplify hardware encoding and decoding.

1) Encoding base reads: Given a stream of bases to compress, Bancroft's algorithm looks for matching regions between the input stream and the compression reference. This is similar to how typical reference-based compression algorithms work. Matching regions are encoded as a 32-bit index into the reference, and unmatched regions are encoded verbatim, using a conventional 2-bit encoding for each base.

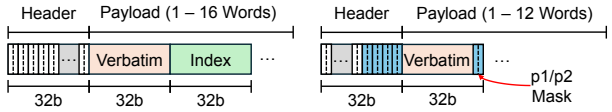
Bancroft's algorithm is unique in that it uses fixed-width 32-bit payloads to encode both types. As a result, verbatim encoding invariably encodes 16 bases. The number of bases in each verbatim encoded field (16 in this case, resulting in 32 bits) is actually a tunable parameter called *stride*, or S.

For matching indices, Bancroft does not encode the *length* of the match, since variable-length matches can complicate the encoding and decoding hardware. Instead, the algorithm is parameterized with a fixed size for each match, where each encoded index represents a fixed-length match of K bases, or K-mer, which is larger than S. Unlike many other algorithms, Bancroft only looks for exact matches, also to simplify hardware.

TABLE I: Sequence encoding header values and their payloads.

Header	Payload	Description
`00`	Y	Verbatim encoding of length S.
`01`	Y	32-bit forward match offset.
`10`	Y	32-bit reverse complement match offset.
`11`	N	Continuation.

The compression encoding supports two more types of matches: First, "reverse complement" match means the K-mer from the input stream matches the reverse complement of a region of the reference. Second, "continuation" means that the match occurs immediately after the previous match in terms of the offset within the compression reference. Forward by K if the previous match was a forward match, and backward by K if the previous match was a reverse complement. Continuations do not require an index to be encoded, meaning the number of payloads encoded per 16-header block will be 16 or fewer. As a result, Higher match rates, especially long sequences of continuation matches, result in better compression. The design space exploration for the optimal selection of K and S is presented in Section VI-C. Table I presents the four possible header values and their corresponding payload descriptions.



(a) Sequence encoding. (b) Quality score encoding.

Fig. 1: Consistent grouped header encoding for sequences and quality scores.

2) Encoding quality scores: To err on the side of correctness, Bancroft implements a lossless compression scheme for quality scores, unlike some advanced tools which implement lossy, quantized compression [29], [30], [69].

The quality score encoding follows a very similar structure with sequence read encoding, using a stream of 32-bit fixed-width fields as presented in Figure 1b. We encode the quality score in the typical 6-bit encoding, meaning five scores can be encoded verbatim in the 32-bit payload with two bits left over. Our quality score encoding does not use a reference, so it does not encode an index into a reference. Instead, it exploits the statistical characteristics of quality scores where the same score is often repeated many times, with little variability. Table II shows the header values and their meaning. Only header `00` has a corresponding payload, encoding a length S array of 6-bit score values. Header `01` specifies that the most recently decoded list of length 5 is repeated again, and `10` means the most recently decoded single score value is repeated 5 times.

TABLE II: Quality encoding header values and their payloads.

Header	Payload	Description
`00`	Y	Verbatim encoding of length S.
`01`	N	Repeats the most recent score of length S.
`10`	N	Replicate the most recent score S times.
`11`	N	Length S string with only $p1$ and $p2$.

The last header, `11`, introduces two new parameters: $p1$ and $p2$. These are the top two most popular quality values in the dataset This header encodes a string consisting only of $p1$ and $p2$ values, and the location of $p2$ values is encoded as a bit mask, encoded using the emph"p1/p2 mask" fields presented in Figure 1b For example, header `11` with bit mask `10101` will result in decoding into `IDIDI`, if $p1$ is `I` and $p2$ is `D`. $p1$ and $p2$ values can be reliably determined via quick sampling, and are typically consistent across reads obtained with the same technology. For HG002, $p1$ and $p2$ are `I` and `D`, taking up 81% and 10% of all occurrences, respectively. This was consistent with every dataset we evaluated across all organisms.

The stream of p1/p2 masks are encoded independently from the payloads they share the 32-bit field with, simply making use of the 2-bit remaining space after encoding five 6-bit values into a 32-bit field. Based on design space exploration, the top four fields of the header are also used to store p1/p2 mask bits, resulting in at most 12 payload fields per block. The first field of a block is always verbatim encoded, to facilitate parallel encoding and decoding by breaking dependency relationships between each encoded block.

B. Compression/Decompression Algorithm

1) Compression of base reads: Figure 2 illustrates how Bancroft compression finds exact matches in the compression reference and translates them to offset values for encoding. Since Bancroft only looks for exact matches to facilitate efficient hardware implementation, we avoid using a complex index structure in favor of an $O(1)$ hash table. The table is pre-constructed from the reference genome to hold the offset of each k-mer within the reference. Such efficient lookup vastly improves Bancroft performance compared to conventional schemes using suffix arrays or alignment algorithms [71], [72].

Since the offset table is constructed offline once for each reference and used repeatedly for the same species, we adopt the cuckoo hash for better space efficiency. Cuckoo hashes use two hash functions per query, so each query can have a choice of avoiding conflicts by selecting one of two locations. Its algorithm ensures that all keys can be accommodated without conflict misses if the capacity of the table is $2\times$ the set size of keys [77]. We note that Bancroft actually uses four hash

functions during compression, although the cuckoo hash is constructed using two. This is because we also look for reverse complemented versions of the target k-mers, as sequencing machines emit both directions of reads. Cuckoo hashes are probabilistic data structures, meaning table lookups can result in false negatives, resulting in fewer matches. However, this does not affect the correctness of the algorithm, and in our experience, it is also quite rare with a realistically large table.

Each of the two offset table lookups represents an offset within the compression reference, each of which is a potential exact match. The algorithm then reads the offset in the compression reference to determine whether any of the two results in an actual exact match. If a match exists, the whole k-mer is encoded compactly as a single offset, and the window moves forward by K. If neither is a match, then the mismatched stride is encoded verbatim, and the window moves forward by S to check the next K-mer. In our eventual prototype, K is 64, and S is 16.

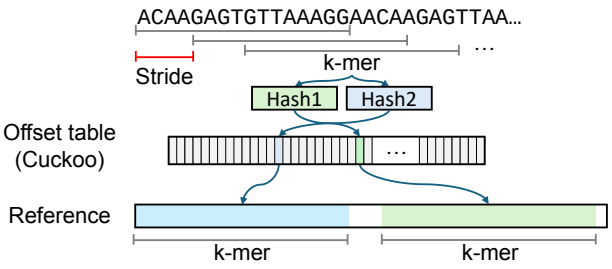


Fig. 2: Reference matching with fixed-k, fixed-stride.

The hardware-accelerated implementation described in Section IV-A also introduces transparent performance enhancements, including probabilistic filtering, to overcome the computation and memory system bottleneck of this algorithm during compression.

2) Compression of Quality Scores: Since quality score compression only looks at the current S values and the previous S values, the compression algorithm trivially implements the encoding scheme described in Section III-A. More details on its hardware implementation are presented in Section IV-A.

3) Decompression of base reads: Thanks to the hardware-aware considerations during encoding, the decompression process of sequence reads is very simple. The decompression software or accelerator simply scans through the header, and decodes the 32-bit words in the payload as verbatim encodings, or as indices to look up the reference genome. If the next element is a continuation, depending on whether the last match or continuation was in forward or reverse order, the next or previous k-mer is fetched from the reference genome. We show in Section IV-B that the grouped header encoding allows very efficient decoding of both verbatim values and efficient reference genome lookups.

4) Decompression of quality scores: Quality score decoding goes through much of the same process. The decompression software or accelerator simply scans through the header, and decodes the 32-bit words in the payload as verbatim encodings, or as repeats of recent values, or as repeated $p1$ values, according to the header. Then, the p1/p2 mask is decoded to change some score values from $p1$ to $p2$. Since the mask bits are encoded in fixed locations, and each mask is uniformly 5 bits, parsing it in hardware is very efficient.

IV. Bancroft Platform Architecture

As illustrated in Figure 3, Bancroft provides a hardware accelerator with a wrapper around large genomic datasets to the user. The primary hardware components in the architecture are the decompression and compression accelerators, while the primary software component is the host-side compressed database. Each compression and decompression accelerator needs access to a fixed-size buffer in HBM, so they must be coupled with fixed-size sets of HBM pseudo-channels (PC). This is described in more detail in the following sections.

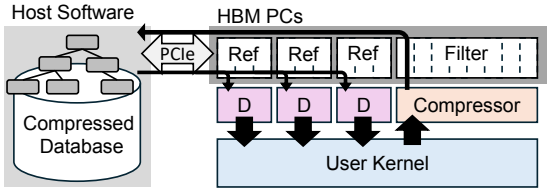


Fig. 3: An example design with three decompressors (D) and one compressor, consuming 17 HBM PCs.

For an FPGA deployment of Bancroft, the number of compressors and decompressors in the system can be configured for each application's requirements, within the physical boundaries of the platform. For example, the prototype's U50 platform has 32 HBM channels, 256 MB each, and the human reference genome requires the capacity of three channels. As a result, up to 10 decompressors can be instantiated.

A. Compression Accelerator

1) Sequence Read Compression: Due to the memory capacity requirement of the cuckoo hash offset table, Bancroft divides the compression process between hardware acceleration and the software manager. For example, the cuckoo hash table to encode the human genome is at least 16 GBs, leaving no space even on medium-range FPGA platforms such as Alveo U55C. Instead, the task of maintaining and querying the offset table, performing the reference comparisons, and encoding the compressed data, is left to the cooperating host software. The hardware accelerator is only responsible for providing a stream of hash values and supplementary information to the software over PCIe. The hardware component is mainly responsible for two tasks: Computing the hash values for the cuckoo hash, and performing probabilistic filtering using a smaller cuckoo hash, to reduce the load on the software.

Figure 4 shows the architecture of the hardware accelerator. The accelerator ingests sequence data either in the form of ASCII-encoded text, such as FASTA and FASTQ, or as a 2-bit binary encoded format generated by user kernels. ASCII input is first encoded in binary format, and a stride shifter

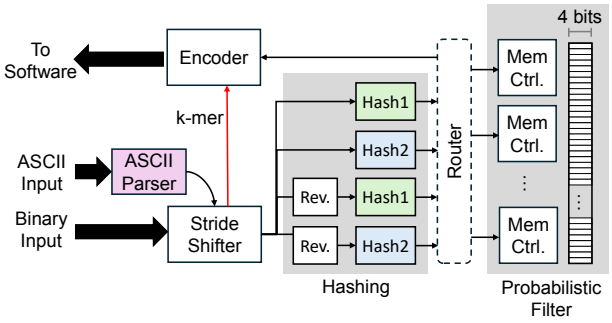


Fig. 4: Hardware portion of the compressor.

emits a stream of k-mers into an array of four pipelined hash function modules. Since our prototype uses a K value of 64, the datapath between the stride shifter and the four hash functions is 128 bits wide. The hardware needs four hash functions because two hashes are needed to query the cuckoo hash, and two more are needed to query the cuckoo hash based on the reverse complement of the k-mer. The computed hash values are independently routed to one of many memory controllers, each interfacing with an HBM PC, to check the probabilistic filter. The filter results, along with the original k-mer as emitted by the stride shifter, are collected at the outbound encoder into a software-friendly format. Bancroft uses the Murmur3 hash function to achieve both high software and hardware throughput.

The probabilistic filter is not part of the algorithm as described in Section III, but a transparent optimization to reduce the number of hash values the software has to verify. This is done via another, much smaller cuckoo hash table in accelerator memory, as depicted in Figure 4. This table is also constructed offline, once for each species. The four hash values point to the same index as they would in the software-side hash table, but instead of storing the 4-byte offset into the reference, each element holds the first 4 bits of the compression reference at this location. If the hash lookup does not match the first 4 bits of the input, we can be sure that the software does not need to check this location. The performance impact of this optimization is quite significant, as presented in Section VI-D. As a result, the compression accelerator needs about 2 GB of HBM memory, as depicted in Figure 3.

Once the likely hash values are received at the software side, it must compare the read k-mer with the potential compression reference k-mers discovered via the cuckoo hash. Since the k-mer the last stride would exist in software already, each shift only needs to send the next 32-bit stride. In our prototype configuration of $K = 64$, this means each stride shift results in 20 bytes of data transfer (4 bytes $\times$4 hash values + 4-byte stride).

The remaining process of compression and data organization in software is described in more detail in Section V.

2) Quality Score Compression: Thanks to the simple, local nature of our quality score compression algorithm, its implementation is also low-overhead and trivially straightforward. Every cycle, 12 header fields are generated by parsing twelve sets of length S quality scores in parallel, determining whether the previous set was repeated, or whether it only consists of $p1$ and $p2$. Since p1/p2 masks are only stored with verbatim fields, the number of verbatim fields is compared with the number of fields with exclusively $p1$ and $p2$ values, to determine whether an insufficient number of p1/p2 mask bits are available. If not, some fields with only $p1$ and $p2$ values also need to be encoded verbatim. Once the verbatim strides are identified, they are compacted down via a pipelined shuffler and then emitted.

B. Decompression Accelerator

The architecture of each decompression accelerator is very simple despite the high performance, thanks to the algorithmic modifications to simplify hardware design. Figure 5 presents the architecture of a compression accelerator. The accelerator scans through both the header and payload of the compressed input using a sliding window of width 4. The accelerator needs to consider a window of slots at once, because the k-mer loaded from memory for each encoded index is only 64 bases, or 128 bits. Sending a memory request for each index would severely degrade performance if the memory bus is wider than this. For example, our FPGA prototype running at 250 MHz needs a 512-bit memory bus to make the best use of memory bandwidth, and transmitting only 128 bits per cycle will waste too much bandwidth. Similarly, verbatim encoded data is stored in units of 32 bits, and decoding one such element per cycle will lead to bandwidth waste.

If the sliding window parser discovers consecutive verbatim encodings, or consecutive continuations, it can either decode up to four verbatim encodings in one cycle, or issue a single 512-bit memory request instead of four requests over four cycles. We emphasize that such efficient design is made possible thanks to the algorithmic modifications. First, having grouped headers allows the parser to quickly determine consecutive elements. Without grouped headers, headers and payloads would be interleaved, meaning the next header can only be decoded after determining whether the current header encodes a continuation or not. Furthermore, fixed encoding width not only makes headers small since length information does not need to be encoded, but it also makes it simple to slide a fixed-width window through both header and payload.

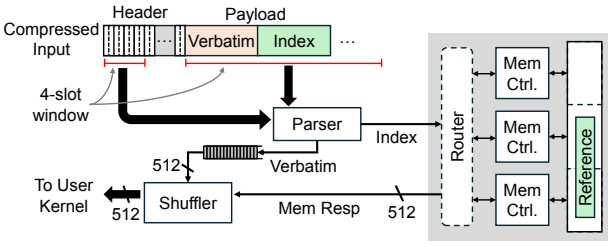


Fig. 5: Decompression hardware accelerator.

Memory read responses and the parsed verbatim elements are collected at the shuffler, which uses a multi-cycle pipelined shifter and a compaction module based on sorting networks to shift and concatenate data across different cycles and remove gaps. The resulting output is a gap-free continuous stream of genomic data, which is naturally expected by the user kernel.

Despite allocating three 250 MB HBM channels per decompressor for the human reference genome, the 512-bit output bus of the decompressor does not become a bottleneck. While three channels at peak bandwidth could theoretically overwhelm the output by $3\times$, our FPGA prototype shows that random reference lookups result in low effective bandwidth, balancing the effective bandwidth. Consequently, reference memory can't be shared between decompressors without performance loss since all channels are kept busy. Section VI-D provides further bandwidth usage details

1) Quality score decompression: Quality score decoding shares much of the same hardware, including 32-bit field extractions and 512-bit shufflers. The mask bits are encoded in fixed locations, and each mask is uniformly 5 bits, making it trivial to parse.

During decoding, all 12 headers are decoded at once, resulting in 60 bytes decoded consistently every cycle. Decoding is done in three pipelined stages. First, verbatim fields are populated by scattering them using the pipelined 512-bit shuffler. Then, repeated $p1$ fields are set, and mask bits are parsed and applied in parallel. Finally, fields that repeat recent values are applied. This way, a decoder always emits 128 decoded bits per cycle, unless the dependency caused by two or more consecutive `01` or `10` headers cause the pipeline to stall. In our experience, such a situation was rare.

V. Software Manager and Interface

The software interface of Bancroft wraps around both the Xilinx XRT environment and the compression functionalities, to provide a familiar and transparent interface to acceleration on both compressed and uncompressed data.

A. Random Access Index

One of the most important features of compressed data management is random access. Downstream processing, such as graph construction based on reads, requires each read to be accessed separately, and the fundamental workload of reference-based alignment requires random access capabilities into the reference.

Bancroft achieves this feature using a B+tree data structure, as hinted at in Figure 3. During compression, a B+tree is constructed with the original file-internal offset as the key. The unit element of insertion and lookup is a 4 KB page of compressed data. In the case of long reads, pages are padded at long read boundaries to facilitate efficient I/O at long read units. This decision is to make it efficient to store data in secondary storage, since PCIe-attached storage such as NVMe can support sufficient bandwidth at a low cost compared to host DRAM.

B. Compression Reference Encoding

The software must also store the reference genome used for compression, because the compression process is split between hardware and software, as described in Section IV-A. The software must perform a lookup into the cuckoo hash table and discover the correct cuckoo hash slot (if any) by comparing the input k-mer against up to four k-mer substrings sampled from the reference genome according to the cuckoo hash lookup.

Bancroft's 2-bit encoded reference format can cause performance overhead when comparing k-mers due to sub-byte addressing. For example, if a k-mer starts from offset 7, with two-bit encoding, the 7th base starts in bit 6 of byte 2. Comparing k-mers in this setting requires repeated shifting operations, which can have performance overhead. To overcome this, Bancroft stores four copies of the reference, each shifted from the original by 2, 4, and 6 bits. During compression, one of the copies are chosen according to the "offset mod 4" value, and then fast `memcmp` can be used since the string would be aligned along byte boundaries. This optimization improves encoding performance on average by $4\times$ compared to either storing byte-aligned ASCII files or on-the-fly shifting.

We emphasize that cuckoo hashes and encoded references need to be created offline only once for each species. Every Bancroft user can simply load and use the encoded references we provide as part of the system.

VI. Evaluation

We evaluated Bancroft using a prototype constructed using an affordable Xilinx U50 Datacenter FPGA accelerator card with 8 GB of 3D-stacked HBM, plugged into either a desktop-class or server-class computer with up to 48 Xeon cores and hundreds of GB of DDR4 memory. The U50 FPGA card is plugged into the host machine via a PCIe Gen3 x16 connection, where our framework was able to achieve a steady-state transfer rate of 9.1 GB/s download and 8.4 GB/s upload.

We expect the insights we obtain from the U50 platform to be representative of more powerful future FPGAs, since the ratio of chip size to HBM bandwidth will likely scale at a similar pace. For example, the U50 delivers 300+ GB/s of peak HBM bandwidth and 0.87 million LUTs, or 344 GB/s/MLUT. More expensive and recent devices, the U55C and V80, have similar ratios, with 353 GB/s/MLUT and 315 GB/s/MLUT, respectively.

A. Resource Utilization

Table III presents the resource utilization of the Bancroft compressor and decompressor modules, each supporting both base reads and quality scores. Thanks to the hardware-aware algorithmic optimizations, the chip overhead of each component is quite small, resulting in the number of HBM channels becoming the primary resource limitation for performance.

TABLE III: Resource Utilization on the U50 FPGA.

Module	LUT	DSP	HBM Channels
Compressor $\times 1$	36K (4.14%)	480 (8%)	8 (25%)
Decompressor $\times 1$	26K (2.99%)	4 (0.07%)	3 (9.3%)

B. Evaluated Datasets

Table IV presents the datasets used to evaluate Bancroft. Datasets include pre-assembled references in the FASTA format without quality scores, as well as long-read datasets in the FASTQ format with quality scores, from various species. We constructed compression references for each species using HG19 [78], GRCm39 [79], and B73 [80] for humans, mouse, and corn, respectively.

TABLE IV: Evaluated datasets

Dataset	Species	Type	Size (GB)
HG16 [78]	Human	Reference	2.67
HG38 [78]	Human	Reference	2.75
HG002 [81]	Human	Long-Read	121.19
HG003 [82]	Human	Long-Read	204.78
HG004 [83]	Human	Long-Read	205.62
mouse-rep1 [84]	Mouse	Long-Read	129.85
mouse-rep2 [85]	Mouse	Long-Read	131.48
maize-B73-rep1 [86]	Maize	Long-Read	130.77

C. Compression Ratio Evaluation

We first present the compression ratios of Bancroft's hardware-optimized genome compression algorithm. The algorithm for compressing nucleotide sequences is defined with two parameters: stride S and K-mer width K. We performed exhaustive problem space exploration to determine the optimal K and S parameters, which we summarize below. We note that all compression ratios are based on the ASCII (8 bits per base) representation.

1) Base read compression: Figure 6 represents the compression efficiency with different S and K configurations, for different datasets. Each dataset uses a compression reference appropriate for its species. For stride evaluation, K is fixed to 64. For K evaluation, S is fixed to 16. Each fixed value is an optimal selection as shown in Figure 6. Bancroft uses S=16 by default, which has a nice further benefit that verbatim encodings are 32-bit, which matches the width of indices into the reference genome, resulting in simpler decoder logic. While smaller strides result in slightly more k-mers being matched to the reference, it also results in more header bits because each verbatim encoding is smaller. Bancroft uses K=64 by default. While larger K can reduce header overhead with clean, pre-assembled reference genomes, it results in lower match rates with long read datasets due to their high error rates. Most experiments with long reads show the best compression at K of 64 (Human, Mouse) and 128 (Maize). We choose the K value of 64, to emphasize its use for Humans.

2) Quality score compression: Figure 7 shows the compression ratios of Bancroft's quality score compression scheme. S was chosen to be 5, since larger values had sharply lower match rates, and smaller values resulted in too much header overhead. $p1$ and $p2$ were `I` and `D` for all datasets.

3) Comparison against state-of-the-art: Figure 8 and 9 evaluate the compression efficiency of Bancroft against other state-of-the-art genome-specific compression tools, for FASTA and FASTQ data formats, respectively. SPRING (2018, [29]) and CoLoRdN (2022, [31]) are reference-free algorithms,

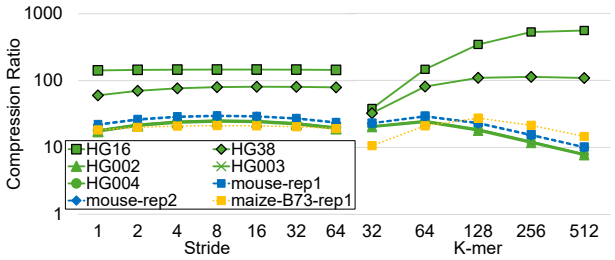


Fig. 6: Sequence compression efficiency with different S and different K (Log-scale).

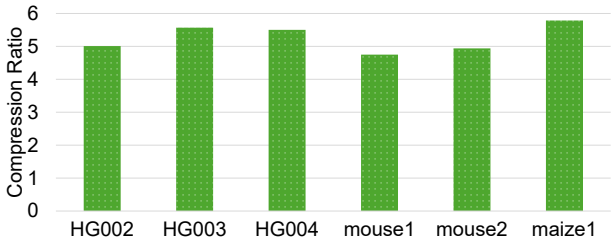


Fig. 7: Quality score compression with different S.

which employ ideas such as read reordering to improve compression. CRAM (2022, [30]) and CoLoRdR (2022, [31]) are reference based algorithms. Bancroft was configured with the default parameter of $S = 16$, $K = 64$.

Figure 8 compares the compression ratios using the FASTA file format, which does not include quality scores, while Figure 9 compares the compression ratios using the FASTQ format, which does include quality scores. We note that all ratios are computed against a 2-bit binary encoding. All systems achieve an additional 4× improvement on average when compared against ASCII encoded formats.

We also note that all compared systems use either quantization or lossy compression for quality scores, as presented in the respective publications. Bancroft only presents lossless compression, to err on the side of fairness. We show competitive compression ratios compared to state-of-the-art, despite its optimizations geared towards efficient hardware.

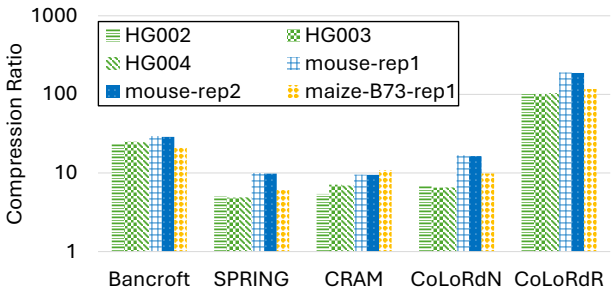


Fig. 8: FASTA compression ratio comparison (Without quality score, Log-scale).

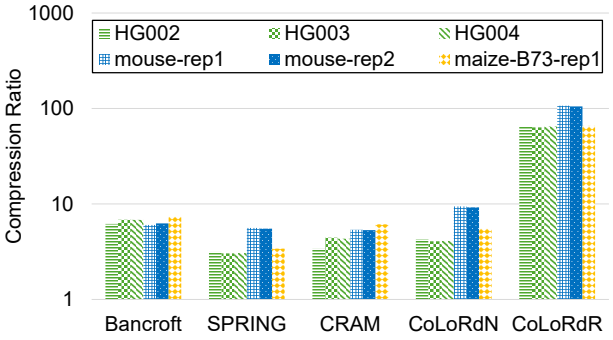


Fig. 9: FASTQ compression ratio comparison (With quality score, Log-scale).

D. Compression/Decompression Performance

1) Decompression performance: Figure 10 presents the throughput of a single decompressor module with different FASTQ datasets. It also demonstrates the minimal bandwidth pressure put on the PCIe to achieve such high decompressor throughput. Thanks to the simple hardware design facilitated by the algorithmic optimizations, each decompressor can very nearly saturate the 512-bit output bus for all datasets tested, while consuming only 3% of the chip space.

2) Realistic ensemble performance: We also present the decompressor performance when working with realistic ensembles of genomic datasets. Figure 11 presents the total bandwidth achieved with Bancroft, when working with multiple datasets in parallel through multiple decompressor channels. The evaluated ensembles are described in Table V. The names of each set describe how many references are used, and how many long read FASTQ datasets are used (e.g., "1R3" means one reference and three long reads). Each set represents a type of analysis. For example, 2R2-1 represents a multi-reference alignment workload with a large read dataset, 1R3-4 represents aligning a very large read dataset against a single reference, and 0R4-1 represents pairwise alignments for De Novo assembly. We note that datasets for different species are not distinguished for this experiment, and are used simply as real-world datasets with different compression characteristics.

Figure 11 presents the total decompression performance across four decompressor modules, for each ensemble. We can see that all configurations reach close to the peak bandwidth of

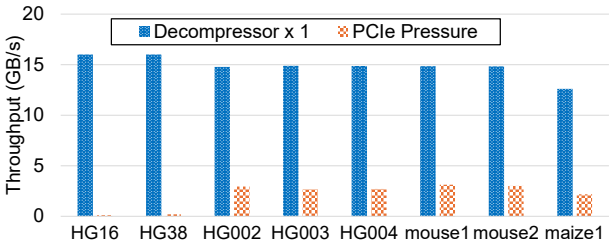


Fig. 10: Single decompressor throughput.

TABLE V: Datasets emulating realistic ensemble workloads.

Set	Composition
2R2-1	HG16+HG38+HG002+HG003
2R2-2	HG16+HG38+HG004+mouse-rep1
1R3-1	HG38+HG002+HG003+HG004
1R3-2	HG38+HG002+mouse-rep1+mouse-rep2
1R3-3	HG38+mouse-rep1+mouse-rep2+maize-B73-rep1
0R4-1	HG002+mouse-rep1+mouse-rep2+maize-B73-rep1

four 512-bit buses (64 GB/s). This number is comparable to the peak DDR4 bandwidth on the higher-end Alveo U250 FPGA platform. More importantly, PCIe does not cause a significant performance degradation, even though some sets do reach the bandwidth limitation.

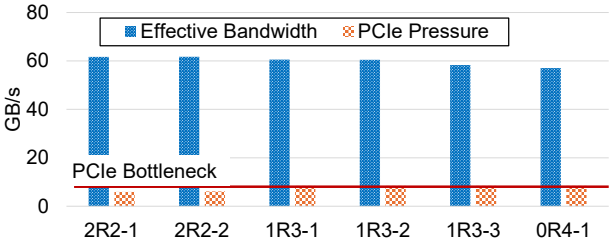


Fig. 11: Ensemble decompressor throughput.

3) Limit study on peak achievable performance: Figure 12 demonstrates one of the most favorable of the realistic workloads for Bancroft, where one long read dataset (HG002) is aligned against multiple reference genomes, in a multi-reference alignment workflow. We notice in Figure 12, that the real-world measured throughput of Bancroft on the U50 stalls around 70 GB/s. This is due to a limitation specific to the U50 platform, where the HBM power is throttled at 7.8 W. In this situation, Bancroft delivers around 34% of the sustained real-world performance of the power-throttled U50 HBM, while accessing unlimited data capacity over PCIe. This measure of performance is similar to the peak memory bandwidth available on the higher-end Alveo U250 FPGA platform equipped with DDR4 memory.

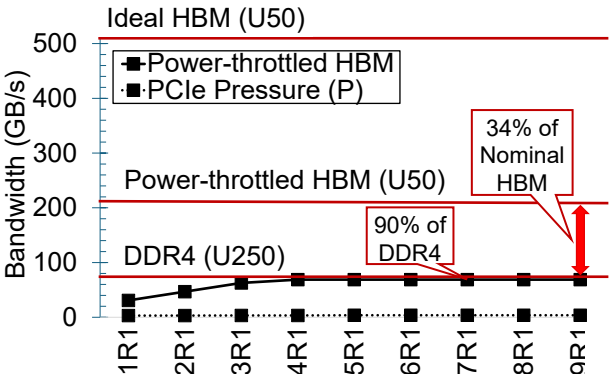


Fig. 12: Bancroft vs. peak HBM bandwidth on power-throttled results.

On devices with no power throttling, e.g., the mid-range Alveo U55C, the available HBM bandwidth is closer to 512 GB/s. On such a device, Bancroft can operate up to ten decompressors in parallel, only limited by the HBM capacity necessary to store the compression reference genome. Across 10 decompressors, Bancroft would achieve 160 GB/s of total throughput, which is 31% of the peak HBM bandwidth, and almost 2× DDR4. This projection is presented in Figure 13.

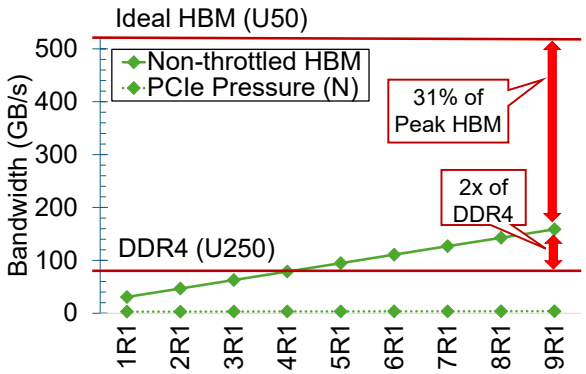


Fig. 13: Bancroft vs. peak HBM bandwidth on non-throttled results.

4) Compression performance: The compressor design partitions work between the host software and the accelerator hardware. Figure 14 compares the performance between different partitionings of work, and how many host CPU threads are necessary to reach maximum performance.

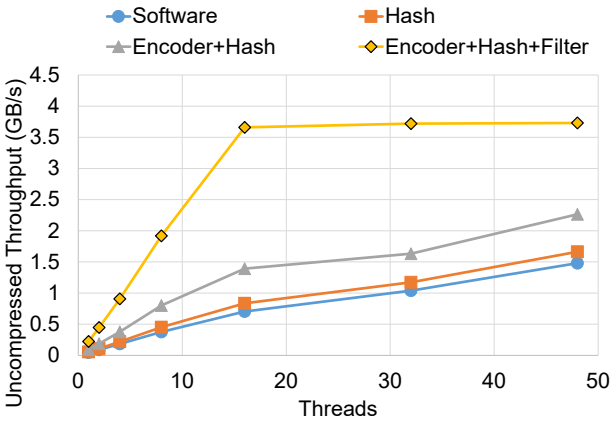


Fig. 14: Compression performance with gradual accelerator offloading on the HG002 long read dataset.

We can see that a pure software implementation ("software") is computation-bound, since every new thread results in better performance. Offloading hashing ("Hash"), ASCII parsing ("Encoder"), or even both ("Encoder+Hash") to hardware did not result in significant performance improvement, even though the only work left for the software at this point is cuckoo hash lookup. We discovered that the 32-bit random access into the cuckoo hash was causing excessive cache misses, turning the host memory system into the bottleneck. Bancroft solves this problem using the probabilistic filter ("Encoder+Hash+Filter"), which reduces host CPU and memory pressure. As a result, Bancroft reaches maximum performance with only 16 CPU threads.

5) Compression accelerator comparisons: The compression throughput of Bancroft is orders of magnitude higher than any existing FPGA genome compression accelerator, as illustrated in Figure 15. We compare against two published genome compression accelerators, Arram15 [71] and Chen23 [72], We note that the FPGA platforms used are different and Bancroft uses the host memory to host the cuckoo hash, rendering raw LUT count comparison inaccurate. However, the trend is clear. Bancroft achieves almost 100× the performance of Arram15, and over 30× Chen23, while using only 3 to 4% of the chip.

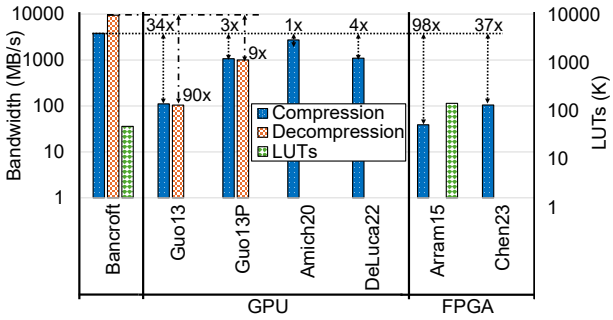


Fig. 15: Comparison of a single Bancroft channel against GPU and FPGA-accelerated compression (Log-scale).

Figure 15 also compares against GPU implementations. To err on the side of fairness, we compare the performance of a single Bancroft channel, based on its worst-case measured performance (B73 Maize). Three systems are compared (Guo13 [73], Amich20 [74], DeLuca22 [75]). The latter two used multiple current-generation GPUs in parallel, so we present performance normalized to a single GPU. Guo13 used a now-outdated K20c, so we also present a scaled projection based on the relative performance against a modern RTX 4090 GPU (Guo13P). Only Guo13 presents a decompression performance, which is similar to its compression performance.

Bancroft achieves superior performance compared to all systems compared against. We also emphasize that Bancroft's decompressor achieves this while consuming only 3% of the U50 chip, while the GPU implementations use all available GPU resources.

E. Case Study: Pre-Alignment Filtering

We provide a case study using pre-alignment filtering, a memory-intensive application crucial for computational genomics. The prevalent seed-and-extend method of genome sequence alignment generates numerous potential matches during seeding, many of which prove unfruitful after alignment. Pre-alignment filtering efficiently reduces computational overhead by quickly eliminating unlikely matches based on

edit distance thresholds. This approach significantly decreases end-to-end work for both long and short reads without compromising accuracy, often achieving an order of magnitude improvement [20], [87]–[89].

The performance of pre-alignment filtering within a real system is often severely memory-bound [20], because every filtering operation requires moving at least one read sequence to the accelerator.

1) Pre-Alignment Filtering Algorithm: We implement a Shifted Hamming Distance (SHD) type filter (e.g., SHD [90], GateKeeper [20], [91], MAGNET [88]), since they have been shown to have good and accurate filtering while highly parallelizable [20]. Figure 16 demonstrates the key idea of SHD filters, filtering based on an edit distance of 1. A filter with an edit distance of one shifts the reference by one to the left and right, resulting in three shifted copies. The read sequence (which may have substitutions, insertions, and deletions) is compared against each copy, to create three-bit masks where "0" and "1" represent a matched base and a non-match, respectively. The three masks are AND'd together, and the number of "1"s remaining becomes the edit distance, which is correctly 2 in this case.

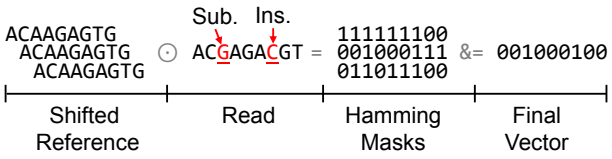


Fig. 16: Shifted Hamming Distance algorithm.

We evaluate this application on Bancroft using a popular configuration with an edit distance of 5, meaning 11 copies are created by shifting the reference from 1 to 5 times in each direction. The filtering kernel processes the 11 shifted references and one read in parallel, using a tree of comparators looking at disjoint windows of 512 bits. A configurable amendment module after the comparator can flexibly apply various optimizations introduced by algorithms such as MAGNET [88]. Then, a tree of saturating 3-bit adders computes the number of 1's in the final vector.

2) Evaluation Against State-of-the-Art Accelerators: We compare the pre-alignment filtering performance on Bancroft against various GPU and FPGA implementations. Evaluated GPU implementations are presented in Table VI, which lists the publication year, GPU platform used, and the memory bandwidth of the GPU platform. Whenever a GPU platform showed different performance between different datasets, we chose the fastest one listed. While the 3080ti is an older-generation GPU, its programmable CUDA core performance (34 TFLOPS) is not drastically lower compared to more modern, but more tensor-centric models such as the H100 (67 TFLOPS). We also note that Gatekeeper implements an SHD-based algorithm, and the others implement other GPU-optimized algorithms. We also compare against ideal FPGA performance, where bandwidth is only bound by accelerator

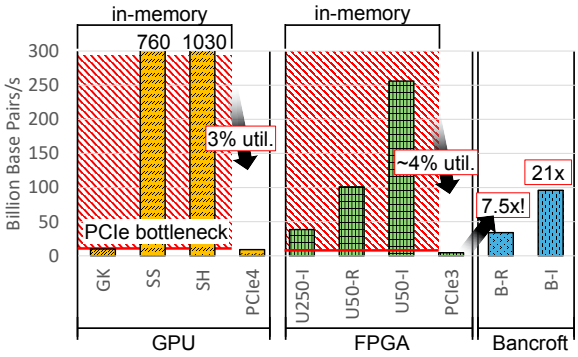


Fig. 17: Bancroft overcomes PCIe limitations to achieve high performance.

memory bandwidth. Specifically, we present U250(I), the ideal performance of the Alveo U250 device with DDR4 memory, as well as U50(R) and U50(I), the U50 device where HBM is power-throttled and not, respectively.

TABLE VI: GPU implementations of pre-alignment filtering.

GateKeeper (GK)	SneakySnake (SS)	SeedHit (SH)
2024 [91]	2020 [89]	2024 [92]
GTX 1080ti	RTX 3080ti	RTX 3080ti
484 GB/s	912 GB/s	912 GB/s

Figure 17 compares the performance of these systems against Bancroft, which implements ten decompressor channels consisting of five pairs of a reference and a long read dataset (HG002). Two performance points are presented, B-R is the measured performance with HBM power throttling, while B-I is the projected unthrottled performance. While many state-of-the-art pre-alignment filtering accelerators achieve very high internal throughput, performance when integrated into an end-to-end system is primarily limited by PCIe performance, resulting in the 3% and 4% utilization numbers presented in the figure.

Thanks to the efficient compression implementation, Bancroft can greatly mitigate this problem within the last-generation PCIe Gen3 performance limitations, improving the effective performance of the pre-alignment filtering accelerator within an end-to-end system by almost an order of magnitude.

VII. Conclusion and Discussion

In this paper, we present Bancroft, a computational genomics accelerator platform which provides access to TBs of genomics data at memory-class performance via hardware-optimized compression. We believe Bancroft is an attractive solution to achieving continued scalability with computational genomics. We distribute our software (C++) and hardware (Bluespec) reference implementations of the Bancroft compression algorithm at https://github.com/SeMinLim/bancroft.

ACKNOWLEDGMENT

This research is partially supported by the PRISM (000705769) center under the JUMP 2.0 program by DARPA/SRC.

This work was also supported by the Institute of Information & Communications Technology Planning & Evaluation(IITP) grant funded by the Korea government(MSIT) (No.RS-2025-02219317, AI Star Fellowship(Kookmin University))

REFERENCES

[1] Z. D. Stephens, S. Y. Lee, F. Faghri, R. H. Campbell, C. Zhai, M. J. Efron, R. Iyer, M. C. Schatz, S. Sinha, and G. E. Robinson, "Big data: astronomical or genomical?," *PLoS biology*, vol. 13, no. 7, p. e1002195, 2015.

[2] National Human Genome Research Insititute, "Dna sequencing costs: Data," 2022.

[3] N. C. for Biotechnology Information, "Genbank and wgs statistics," 2024.

[4] F. Chen, L. Song, Y. Chen, *et al.*, "Parc: A processing-in-cam architecture for genomic long read pairwise alignment using reram," in *2020 25th Asia and South Pacific Design Automation Conference (ASP-DAC)*, pp. 175–180, IEEE, 2020.

[5] R. Kaplan, L. Yavits, and R. Ginosasr, "Bioseal: In-memory biological sequence alignment accelerator for large-scale genomic data," in *Proceedings of the 13th ACM International Systems and Storage Conference*, pp. 36–48, 2020.

[6] Y. Turakhia, G. Bejerano, and W. J. Dally, "Darwin: A genomics coprocessor provides up to 15,000 x acceleration on long read assembly," *ACM SIGPLAN Notices*, vol. 53, no. 2, pp. 199–213, 2018.

[7] C. E. Leiserson, N. C. Thompson, J. S. Emer, B. C. Kuszmaul, B. W. Lampson, D. Sanchez, and T. B. Schardl, "There's plenty of room at the top: What will drive computer performance after moore's law?," *Science*, vol. 368, no. 6495, p. eaam9744, 2020.

[8] Y. Kim, "Architectural techniques to enhance dram scaling," *Ph. D. dissertation, Carnegie Mellon University*, 2015.

[9] S. Shiratake, "Scaling and performance challenges of future dram," in *2020 IEEE international memory workshop (IMW)*, pp. 1–3, IEEE, 2020.

[10] K. Kim and M.-j. Park, "Present and future, challenges of high bandwith memory (hbm)," in *2024 IEEE International Memory Workshop (IMW)*, pp. 1–4, IEEE, 2024.

[11] B. Hyun, T. Kim, D. Lee, and M. Rhu, "Pathfinding future pim architectures by demystifying a commercial pim technology," in *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pp. 263–279, IEEE, 2024.

[12] S. S. Banerjee, M. El-Hadedy, J. B. Lim, Z. T. Kalbarczyk, D. Chen, S. S. Lumetta, and R. K. Iyer, "Asap: Accelerated short-read alignment on programmable hardware," *IEEE Transactions on Computers*, vol. 68, no. 3, pp. 331–346, 2018.

[13] H. M. Waidyasooriya and M. Hariyama, "Hardware-acceleration of short-read alignment based on the burrows-wheeler transform," *IEEE Transactions on Parallel and Distributed Systems*, vol. 27, no. 5, pp. 1358–1372, 2015.

[14] J. S. Kim, D. Senol Cali, H. Xin, D. Lee, S. Ghose, M. Alser, H. Hassan, O. Ergin, C. Alkan, and O. Mutlu, "Grim-filter: Fast seed location filtering in dna read mapping using processing-in-memory technologies," *BMC genomics*, vol. 19, pp. 23–40, 2018.

[15] O. Mutlu, "Accelerating genome analysis: A primer on an ongoing journey," *Keynote Talk at AACBB*, 2019.

[16] J. Zhang, M. Kwon, H. Kim, H. Kim, and M. Jung, "Flashgpu: Placing new flash next to gpu cores," in *Proceedings of the 56th Annual Design Automation Conference 2019*, pp. 1–6, 2019.

[17] A. Dhar, S. Huang, J. Xiong, D. Jamsek, B. Mesnet, J. Huang, N. S. Kim, W.-m. Hwu, and D. Chen, "Near-memory and in-storage fpga acceleration for emerging cognitive computing workloads," in *2019 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*, pp. 68–75, IEEE, 2019.

[18] D. Lee, A. Chang, M. Ahn, J. Gim, J. Kim, J. Jung, K.-W. Choi, V. Pham, O. Rebholz, K. T. Malladi, *et al.*, "Optimizing data movement with near-memory acceleration of in-memory dbms.," in *EDBT*, pp. 371–374, 2020.

[19] G. Jonatan, H. Cho, H. Son, X. Wu, N. Livesay, E. Mora, K. Shivdikar, J. L. Abellán, A. Joshi, D. Kaeli, *et al.*, "Scalability limitations of processing-in-memory using real system evaluations," *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, vol. 8, no. 1, pp. 1–28, 2024.

[20] M. Alser, H. Hassan, H. Xin, O. Ergin, O. Mutlu, and C. Alkan, "Gatekeeper: a new hardware architecture for accelerating pre-alignment in dna short read mapping," *Bioinformatics*, vol. 33, no. 21, pp. 3355–3363, 2017.

[21] M. Chen, C. Liu, S. Liang, L. He, Y. Wang, L. Zhang, H. Li, and X. Li, "An energy-efficient in-memory accelerator for graph construction and updating," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 43, no. 6, pp. 1781–1793, 2024.

[22] M. H.-Y. Fritz, R. Leinonen, G. Cochrane, and E. Birney, "Efficient storage of high throughput dna sequencing data using reference-based compression," *Genome research*, vol. 21, no. 5, pp. 734–740, 2011.

[23] B. Chern, I. Ochoa, A. Manolakos, A. No, K. Venkat, and T. Weissman, "Reference based genome compression," in *2012 IEEE Information Theory Workshop*, pp. 427–431, IEEE, 2012.

[24] Y.-T. Chen, J. Cong, Z. Fang, J. Lei, and P. Wei, "When spark meets {FPGAs}: A case study for {Next-Generation}{DNA} sequencing acceleration," in *8th USENIX Workshop on Hot Topics in Cloud Computing (HotCloud 16)*, 2016.

[25] X. Fei, Z. Dan, L. Lina, M. Xin, and Z. Chunlei, "Fpgasw: accelerating large-scale smith–waterman sequence alignment application with backtracking on fpga linear systolic array," *Interdisciplinary Sciences: Computational Life Sciences*, vol. 10, pp. 176–188, 2018.

[26] E. Rucci, C. Garcia, G. Botella, A. De Giusti, M. Naiouf, and M. Prieto-Matias, "Swifold: Smith-waterman implementation on fpga with opencl for long dna sequences," *BMC systems biology*, vol. 12, pp. 43–53, 2018.

[27] L. Wu, D. Bruns-Smith, F. A. Nothaft, Q. Huang, S. Karandikar, J. Le, A. Lin, H. Mao, B. Sweeney, K. Asanović, *et al.*, "Fpga accelerated indel realignment in the cloud," in *2019 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pp. 277–290, IEEE, 2019.

[28] D. Fujiki, S. Wu, N. Ozog, K. Goliya, D. Blaauw, S. Narayanasamy, and R. Das, "Seedex: A genome sequencing accelerator for optimal alignments in subminimal space," in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pp. 937–950, IEEE, 2020.

[29] S. Chandak, K. Tatwawadi, I. Ochoa, M. Hernaez, and T. Weissman, "Spring: a next-generation compressor for fastq data," *Bioinformatics*, vol. 35, no. 15, pp. 2674–2676, 2018.

[30] J. K. Bonfield, "Cram 3.1: advances in the cram file format," *Bioinformatics*, vol. 38, no. 6, pp. 1497–1503, 2022.

[31] M. Kokot, A. Gudyś, H. Li, and S. Deorowicz, "Colord: compressing long reads," *Nature methods*, vol. 19, no. 4, pp. 441–444, 2022.

[32] H. Nakagawa and M. Fujita, "Whole genome sequencing analysis for cancer genomics and precision medicine," *Cancer science*, vol. 109, no. 3, pp. 513–522, 2018.

[33] S. H. Gage, G. Davey Smith, J. J. Ware, J. Flint, and M. R. Munafo, "G= e: What gwas can tell us about the environment," *PLoS genetics*, vol. 12, no. 2, p. e1005765, 2016.

[34] E. Cano-Gamez and G. Trynka, "From gwas to function: using functional genomics to identify the mechanisms underlying complex diseases," *Frontiers in genetics*, vol. 11, p. 424, 2020.

[35] R. K. Varshney, A. Bohra, J. Yu, A. Graner, Q. Zhang, and M. E. Sorrells, "Designing future crops: genomics-assisted breeding comes of age," *Trends in Plant Science*, vol. 26, no. 6, pp. 631–649, 2021.

[36] A. Sawyer, T. Free, and J. Martin, "Metagenomics: preventing future pandemics," 2021.

[37] V. Racaniello, "Moving beyond metagenomics to find the next pandemic virus," *Proceedings of the National Academy of Sciences*, vol. 113, no. 11, pp. 2812–2814, 2016.

[38] G. S. Ginsburg and H. F. Willard, "Genomic and personalized medicine: foundations and applications," *Translational research*, vol. 154, no. 6, pp. 277–287, 2009.

[39] K. Offit, "Personalized medicine: new genomics, old lessons," *Human genetics*, vol. 130, pp. 3–14, 2011.

[40] E. A. Ashley, "Towards precision medicine," *Nature Reviews Genetics*, vol. 17, no. 9, pp. 507–522, 2016.

[41] M. J. Chaisson, R. K. Wilson, and E. E. Eichler, "Genetic variation and the de novo assembly of human genomes," *Nature Reviews Genetics*, vol. 16, no. 11, pp. 627–640, 2015.

[42] A. Zeni, G. Guidi, M. Ellis, N. Ding, M. D. Santambrogio, S. Hofmeyr, A. Buluç, L. Oliker, and K. Yelick, "Logan: High-performance gpu-based x-drop long-read alignment," in *2020 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pp. 462–471, IEEE, 2020.

[43] H. Cheng, Y. Zhang, and Y. Xu, "Bitmapper2: A gpu-accelerated all-mapper based on the sparse q-gram index," *IEEE/ACM Transactions on Computational Biology and Bioinformatics*, vol. 16, no. 3, pp. 886–897, 2018.

[44] N. Cadenelli, J. Polo, and D. Carrera, "Accelerating k-mer frequency counting with gpu and non-volatile memory," in *2017 IEEE 19th International Conference on High Performance Computing and Communications; IEEE 15th International Conference on Smart City; IEEE 3rd International Conference on Data Science and Systems (HPCC/SmartCity/DSS)*, pp. 434–441, IEEE, 2017.

[45] K. Liyanage, H. Samarakoon, S. Parameswaran, and H. Gamaarachchi, "Efficient end-to-end long-read sequence mapping using minimap2-fpga integrated with hardware accelerated chaining," *Scientific Reports*, vol. 13, no. 1, p. 20174, 2023.

[46] A. Haghi, S. Marco-Sola, L. Alvarez, D. Diamantopoulos, C. Hagleitner, and M. Moreto, "An fpga accelerator of the wavefront algorithm for genomics pairwise alignment," in *2021 31st International Conference on Field-Programmable Logic and Applications (FPL)*, pp. 151–159, IEEE, 2021.

[47] W. Qiao, Z. Fang, M.-C. F. Chang, and J. Cong, "An fpga-based bwt accelerator for bzip2 data compression," in *2019 IEEE 27th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, pp. 96–99, IEEE, 2019.

[48] T. J. Ham, D. Bruns-Smith, B. Sweeney, Y. Lee, S. H. Seo, U. G. Song, Y. H. Oh, K. Asanovic, J. W. Lee, and L. W. Wills, "Genesis: A hardware acceleration framework for genomic data analysis," in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, pp. 254–267, 2020.

[49] Y. Turakhia, S. D. Goenka, G. Bejerano, and W. J. Dally, "Darwin-wga: A co-processor provides increased sensitivity in whole genome alignments with high speedup," in *2019 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pp. 359–372, IEEE, 2019.

[50] D. Fujiki, A. Subramaniyan, T. Zhang, Y. Zeng, R. Das, D. Blaauw, and S. Narayanasamy, "Genax: A genome sequencing accelerator," in *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*, pp. 69–82, IEEE, 2018.

[51] D. S. Cali, K. Kanellopoulos, J. Lindegger, Z. Bingöl, G. S. Kalsi, Z. Zuo, C. Firtina, M. B. Cavlak, J. Kim, N. M. Ghiasi, *et al.*, "Segram: A universal hardware accelerator for genomic sequence-to-graph and sequence-to-sequence mapping," in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, pp. 638–655, 2022.

[52] D. S. Cali, G. S. Kalsi, Z. Bingöl, C. Firtina, L. Subramanian, J. S. Kim, R. Ausavarungnirun, M. Alser, J. Gomez-Luna, A. Boroumand, *et al.*, "Genasm: A high-performance, low-power approximate string matching acceleration framework for genome sequence analysis," in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pp. 951–966, IEEE, 2020.

[53] Z. Jahshan, I. Merlin, E. Garzón, and L. Yavits, "Dash-cam: dynamic approximate search content addressable memory for genome classification," in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, pp. 1453–1465, 2023.

[54] Z. Zou, H. Chen, P. Poduval, Y. Kim, M. Imani, E. Sadredini, R. Cammarota, and M. Imani, "Biohd: an efficient genome sequence search platform using hyperdimensional memorization," in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, pp. 656–669, 2022.

[55] L. Chen, J. Zhu, G. Peng, M. Liu, S. Wei, and L. Liu, "Gem: Ultra-efficient near-memory reconfigurable acceleration for read mapping by dividing and predictive scattering," *IEEE Transactions on Parallel and Distributed Systems*, 2023.

[56] G. D. Varsamis, I. G. Karafyllidis, K. Gilkes, U. Arranz, R. Martin-Cuevas, G. Calleja, P. Dimitrakis, P. Kolovos, R. Sandaltzopoulos, H. Jessen, *et al.*, "Quantum gate algorithm for reference-guided dna sequence alignment," *Computational Biology and Chemistry*, vol. 107, p. 107959, 2023.

[57] N. Mansouri Ghiasi, J. Park, H. Mustafa, J. Kim, A. Olgun, A. Gollwitzer, D. Senol Cali, C. Firtina, H. Mao, N. Almadhoun Alserr, *et al.*, "Genstore: A high-performance in-storage processing system for genome sequence analysis," in *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, pp. 635–654, 2022.

[58] J. Cheng, L. Hu, W. Xu, H. Chen, and T. Xia, "Hardware acceleration of minimap2 genomic sequence alignment algorithm," in *Proceedings of the 53rd International Conference on Parallel Processing*, (New York, NY, USA), p. 887–897, Association for Computing Machinery, 2024.

[59] L. Guo, J. Lau, Z. Ruan, P. Wei, and J. Cong, "Hardware acceleration of long read pairwise overlapping in genome sequencing: A race between fpga and gpu," in *2019 IEEE 27th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, pp. 127–135, 2019.

[60] S.-M. Lim, E. Aliaj, and S.-W. Jun, "Morbius: Platform-adaptive hardware accelerator for scalable sequence motif discovery," in *2024 IEEE 20th International Conference on e-Science (e-Science)*, pp. 1–10, IEEE, 2024.

[61] P. Levi, "It's the end of dram as we know it," 2023.

[62] T. Ziegler, C. Binnig, and V. Leis, "Scalestore: A fast and cost-efficient storage engine using dram, nvme, and rdma," in *Proceedings of the 2022 International Conference on Management of Data*, pp. 685–699, 2022.

[63] D. Gouk, M. Kwon, H. Bae, S. Lee, and M. Jung, "Memory pooling with cxl," *IEEE Micro*, vol. 43, no. 2, pp. 48–57, 2023.

[64] W. Huangfu, K. T. Malladi, A. Chang, and Y. Xie, "Beacon: Scalable near-data-processing accelerators for genome analysis near memory pool with the cxl support," in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pp. 727–743, IEEE, 2022.

[65] D. Mansouri, X. Yuan, and A. Saidani, "A new lossless dna compression algorithm based on a single-block encoding scheme," *Algorithms*, vol. 13, no. 4, p. 99, 2020.

[66] M. Silva, D. Pratas, and A. J. Pinho, "Efficient dna sequence compression with neural networks," *GigaScience*, vol. 9, no. 11, p. giaa119, 2020.

[67] W. Cui, Z. Yu, Z. Liu, G. Wang, and X. Liu, "Compressing genomic sequences by using deep learning," in *International Conference on Artificial Neural Networks*, pp. 92–104, Springer, 2020.

[68] K. Sheena and M. S. Nair, "Gencoder: A novel convolutional neural network based autoencoder for genomic sequence data compression," *IEEE/ACM Transactions on Computational Biology and Bioinformatics*, 2024.

[69] I. Ochoa, M. Hernaez, R. Goldfeder, T. Weissman, and E. Ashley, "Effect of lossy compression of quality scores on variant calling," *Briefings in bioinformatics*, vol. 18, no. 2, pp. 183–194, 2017.

[70] Ł. Roguski, I. Ochoa, M. Hernaez, and S. Deorowicz, "Fastore: a space-saving solution for raw sequencing data," *Bioinformatics*, vol. 34, no. 16, pp. 2748–2756, 2018.

[71] J. Arram, M. Pflanzer, T. Kaplan, and W. Luk, "Fpga acceleration of reference-based compression for genomic data," in *2015 International Conference on Field Programmable Technology (FPT)*, pp. 9–16, IEEE, 2015.

[72] S. Chen, Y. Chen, Z. Wang, W. Qin, J. Zhang, H. Nand, J. Zhang, J. Li, X. Zhang, X. Liang, *et al.*, "Efficient sequencing data compression and fpga acceleration based on a two-step framework," *Frontiers in Genetics*, vol. 14, p. 1260531, 2023.

[73] G. Guo, S. Qiu, Z. Ye, B. Wang, L. Fang, M. Lu, S. See, and R. Mao, "Gpu-accelerated adaptive compression framework for genomics data," in *2013 IEEE International Conference on Big Data*, pp. 181–186, IEEE, 2013.

[74] M. Amich, P. D. Luca, and S. Fiscale, "Accelerated implementation of fqsqueezer novel genomic compression method," in *2020 19th International Symposium on Parallel and Distributed Computing (ISPDC)*, pp. 158–163, 2020.

[75] P. De luca, A. Di Mauro, and S. Fiscale, *On Next-Generation Sequencing Compression via Multi-GPU*, pp. 457–466. 05 2022.

[76] J. Dean *et al.*, "Challenges in building large-scale information retrieval systems," in *Keynote of the 2nd ACM international conference on web search and data mining (WSDM)*, vol. 10, 2009.

[77] R. Pagh and F. F. Rodler, "Cuckoo hashing," *Journal of Algorithms*, vol. 51, no. 2, pp. 122–144, 2004.

[78] B. J. Raney, G. P. Barber, A. Benet-Pagès, J. Casper, H. Clawson, M. S. Cline, M. Diekhans, C. Fischer, J. Navarro Gonzalez, G. Hickey, *et al.*, "The ucsc genome browser database: 2024 update," *Nucleic Acids Research*, vol. 52, no. D1, pp. D1082–D1088, 2024.

[79] N. L. of Medicine, "Genome assembly grcm39," 2020.

[80] N. L. of Medicine, "Genome assembly b73 refgen_v4," 2017.

[81] PacBio, "Pacbio hifi long read hg002," 2024.
[82] PacBio, "Pacbio hifi long read hg003," 2024.
[83] PacBio, "Pacbio hifi long read hg004," 2024.
[84] PacBio, "Pacbio hifi long read mouse-rep1," 2024.
[85] PacBio, "Pacbio hifi long read mouse-rep2," 2024.
[86] PacBio, "Pacbio hifi long read maize-b73-rep1," 2024.
[87] M. Alser, H. Hassan, A. Kumar, O. Mutlu, and C. Alkan, "Shouji: a fast and efficient pre-alignment filter for sequence alignment," *Bioinformatics*, vol. 35, no. 21, pp. 4255–4263, 2019.
[88] M. Alser, O. Mutlu, and C. Alkan, "Magnet: understanding and improving the accuracy of genome pre-alignment filtering," *arXiv preprint arXiv:1707.01631*, 2017.
[89] M. Alser, T. Shahroodi, J. Gómez-Luna, C. Alkan, and O. Mutlu, "Sneakysnake: a fast and accurate universal genome pre-alignment filter for cpus, gpus and fpgas," *Bioinformatics*, vol. 36, no. 22-23, pp. 5282–5290, 2020.
[90] H. Xin, J. Greth, J. Emmons, G. Pekhimenko, C. Kingsford, C. Alkan, and O. Mutlu, "Shifted hamming distance: a fast and accurate simd-friendly filter to accelerate alignment verification in read mapping," *Bioinformatics*, vol. 31, no. 10, pp. 1553–1560, 2015.
[91] Z. Bingöl, M. Alser, O. Mutlu, O. Ozturk, and C. Alkan, "Gatekeeper-gpu: Fast and accurate pre-alignment filtering in short read mapping," *IEEE Transactions on Computers*, 2024.
[92] Z. Ju, J. Zhang, X. Li, J. Meng, and Y. Wei, "Seedhit: a gpu friendly pre-align filtering algorithm," *IEEE/ACM Transactions on Computational Biology and Bioinformatics*, 2024.